MathSnap-AI

Compact Handwritten Mathematical Expression Recognition
with Coverage-Aware Attention Refinement

MathSnap-AI Project

March 2026

Abstract

MathSnap-AI is an end-to-end system for Handwritten Mathematical Expression Recognition (HMER) that converts photographs of handwritten mathematics into LaTeX markup. The system is built on **Mini-CoMER**, a compact encoder–decoder neural network with 6.39 M parameters—a 68% reduction from the original CoMER architecture. Mini-CoMER pairs a DenseNet-based feature extractor with a Transformer decoder augmented by an Attention Refinement Module (ARM) that prevents redundant spatial attention. Trained on the CROHME dataset, the model achieves an expression recognition rate (ExpRate) of 47.12% while maintaining real-time inference capability on consumer-grade GPUs. The full system comprises a FastAPI backend deployed on Hugging Face Spaces and a React frontend hosted on Vercel.

Keywords: handwritten math recognition, encoder–decoder, attention refinement, DenseNet, Transformer, LaTeX generation

Contents

1 Introduction

Handwritten Mathematical Expression Recognition (HMER) is the task of converting an image of a handwritten mathematical expression into a structured representation such as LaTeX. Unlike standard OCR, HMER must capture two-dimensional spatial relationships—superscripts, subscripts, fractions, and nested structures—making it substantially more challenging.

MathSnap-AI addresses this problem with three objectives:

1. **Compactness.** Reduce model size to enable deployment on resource-constrained hardware without sacrificing recognition quality.
2. **Coverage awareness.** Prevent the decoder from repeatedly attending to dominant symbols while ignoring others, a common failure mode in attention-based sequence generation.
3. **Deployability.** Provide a complete, production-ready system with a web frontend, a REST API, and automated deployment.

The resulting model, **Mini-CoMER**, contains 6.39 M parameters (68% fewer than the original CoMER) and runs inference in real time on a single consumer GPU.

2 Related Work

Encoder–decoder approaches. Early HMER systems used CNN encoders paired with RNN decoders with attention. WAP introduced a coverage-based attention mechanism to address the under-attention problem. BTTR replaced the RNN decoder with a bidirectional Transformer, improving parallelism during training.

CoMER. The Coverage-augmented Multi-head attention Enhanced Recognizer (CoMER) integrated an Attention Refinement Module (ARM) into multi-head cross-attention layers, explicitly tracking cumulative attention coverage. While effective, CoMER's ∼20 M parameters made deployment on edge devices impractical.

Model compression for HMER. Prior work has explored knowledge distillation and channel pruning for optical character recognition, but few studies target HMER-specific architectures. Mini-CoMER takes a direct approach: narrowing the DenseNet growth rate and reducing decoder depth while retaining the ARM mechanism that is critical for expression-level accuracy.

3 Dataset

3.1 CROHME

The Competition on Recognition of Online Handwritten Mathematical Expressions (CROHME) dataset is the standard benchmark for HMER. MathSnap-AI uses the merged CROHME 2014/2016/2019 splits:

Table 1: CROHME dataset split distribution.

Split	Samples	Proportion
Train	21,855	80.7%
Validation	1,702	6.3%
Test	3,499	12.9%
Total	**27,056**	**100%**

3.2 Vocabulary

The vocabulary contains 114 tokens: 4 special tokens (<PAD>, <SOS>, <EOS>, <UNK>) and 110 LaTeX tokens spanning digits, Latin and Greek letters, operators, functions, structural commands, delimiters, and mathematical symbols.

Table 2: Vocabulary composition.

Category	Examples
Special (4)	<PAD>, <SOS>, <EOS>, <UNK>
Digits (10)	0–9
Letters (37)	a–z, A, B, C, ...
Greek (10)	\alpha, \beta, \gamma, \theta, \pi, ...
Operators (17)	+, -, =, \times, \leq, \geq, ...
Functions (5)	\sin, \cos, \tan, \log, \lim
Structural (8)	\frac, \sqrt, ^, _, {, }
Delimiters (5)	(,), [,], |
Symbols (18)	\infty, \sum, \int, \forall, \exists, ...

3.3 Sequence Length Distribution

Target sequences have a mean length of ∼12 tokens (median ∼10), with 95% of samples below 30 tokens and a hard maximum of 200 tokens.

4 Model Architecture

Mini-CoMER follows an encoder–decoder design with three main components: a DenseNet encoder, a Transformer decoder, and an Attention Refinement Module (ARM).

4.1 Architecture Overview

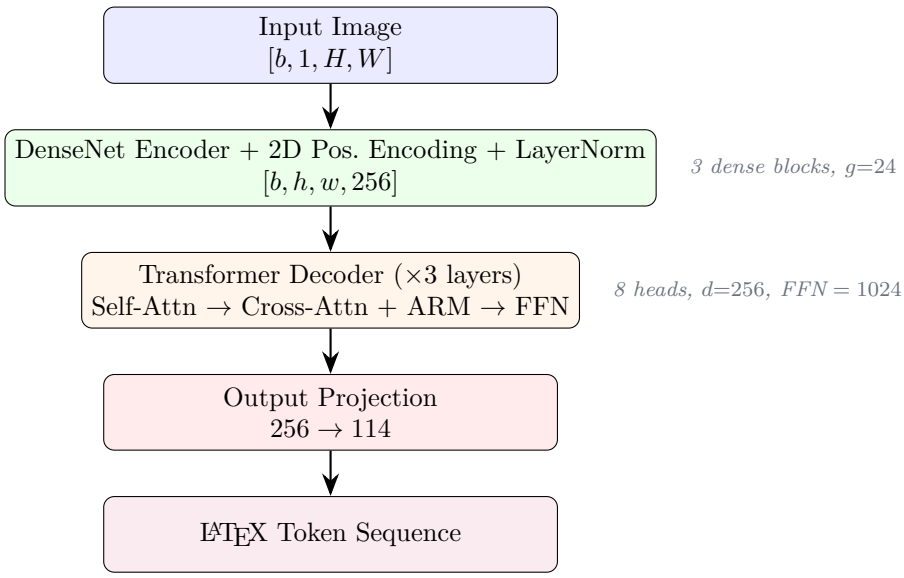

Figure 1: Mini-CoMER architecture overview.

4.2 DenseNet Encoder

The encoder extracts spatial features from a grayscale input image using a DenseNet-B backbone with three dense blocks.

Table 3: Encoder layer configuration.

Stage	**Operation**	**Output Shape**
Input	—	$[b, 1, H, W]$
Initial conv	Conv(1→48, 7×7, s=2) + BN + ReLU	$[b, 48, H/2, W/2]$
Max pool	2×2, s=2	$[b, 48, H/4, W/4]$
Dense Block 1	16 × Bottleneck(g=24)	$[b, 432, H/4, W/4]$
Transition 1	Conv(1×1, r=0.5) + AvgPool	$[b, 216, H/8, W/8]$
Dense Block 2	16 × Bottleneck(g=24)	$[b, 600, H/8, W/8]$
Transition 2	Conv(1×1, r=0.5) + AvgPool	$[b, 300, H/16, W/16]$
Dense Block 3	16 × Bottleneck(g=24)	$[b, 684, H/16, W/16]$
Projection	Conv(684→256, 1×1)	$[b, 256, h, w]$
Pos. encoding	2D sinusoidal	$[b, h, w, 256]$
LayerNorm	—	$[b, h, w, 256]$

Each bottleneck layer consists of a 1×1 convolution expanding to $4g = 96$ channels followed by a 3×3 convolution producing $g = 24$ new feature channels. Dense connections concatenate the output of every layer to all subsequent layers within a block.

4.2.1 2D Positional Encoding

Standard Transformer positional encodings are one-dimensional. Since HMER requires awareness of both horizontal and vertical position (e.g., superscripts vs. subscripts), the encoder applies a 2D sinusoidal encoding:

$$\mathrm{PE}_{(y,x,2i)} = \sin\left(\frac{y}{T^{2i/d}}\right), \qquad \mathrm{PE}_{(y,x,2i+1)} = \cos\left(\frac{y}{T^{2i/d}}\right) \tag{1}$$

where $T = 10{,}000$ is the temperature, $d = 256$ is the model dimension, and analogous terms are computed for the x coordinate. Positions are derived from cumulative masks to handle variable-size images.

4.3 Transformer Decoder

The decoder autoregressively generates LaTeX tokens conditioned on the encoder features. It consists of a word embedding layer, 1D positional encoding, three Transformer decoder layers, and a linear output projection.

Table 4: Decoder hyperparameters.

Parameter	Value
Number of layers	3
Attention heads	8
Model dimension (d)	256
Head dimension	32
FFN inner dimension	1,024
Dropout	0.3
Max sequence length	200
Vocabulary size	114

Each decoder layer performs three sub-operations with residual connections and layer normalization:

1. **Causal self-attention** over the target sequence, preventing tokens from attending to future positions.
2. **Cross-attention** to the encoder feature map, binding each output token to spatial image regions. The ARM (Section 4.4) intervenes here.
3. **Position-wise feed-forward network** ($256 \to 1024 \to 256$) with ReLU activation.

4.4 Attention Refinement Module (ARM)

The ARM addresses the *coverage problem*: without explicit tracking, the decoder tends to repeatedly attend to visually salient symbols while ignoring less prominent ones, causing missed or duplicated tokens.

4.4.1 Mechanism

Let $\alpha_t \in \mathbb{R}^{n_h \times S}$ denote the cross-attention weights at decoding step t across $n_h = 8$ heads over $S = h \times w$ spatial positions. The ARM computes a coverage bias β_t as follows:

$$C_t = \sum_{k=1}^{t-1} \alpha_k \tag{2}$$

$$\hat{C}_t = \mathrm{Conv}_{5\times5}(\mathrm{reshape}(C_t)) \tag{3}$$

$$\beta_t = \mathrm{MaskBN}\Big(\mathrm{Conv}_{1\times1}\Big(\mathrm{ReLU}(\hat{C}_t)\Big)\Big) \tag{4}$$

The coverage bias β_t is subtracted from the cross-attention logits before softmax, discouraging re-attention to already-covered regions.

4.4.2 Configuration

Table 5: ARM hyperparameters.

Parameter	Value
Intermediate channels (d_c)	32
Coverage kernel size	5×5
Activation	ReLU
Post-normalization	Masked BatchNorm
Cross-coverage	Enabled
Self-coverage	Enabled

Masked BatchNorm applies normalization only over valid (non-padded) spatial positions, preventing padding artifacts from corrupting batch statistics.

4.5 Parameter Budget

Table 6: Parameter breakdown: Mini-CoMER vs. original CoMER.

Component	Mini-CoMER	CoMER
DenseNet Encoder	∼4.2 M	∼13 M
Transformer Decoder	∼2.1 M	∼6.5 M
ARM	∼62 K	∼170 K
Total	**6.39 M**	**∼20 M**

The 68% reduction is achieved by narrowing the DenseNet growth rate from 48 to 24, reducing decoder layers from 6 to 3, and lowering the FFN dimension from 2048 to 1024.

5 Preprocessing

Input images undergo a deterministic preprocessing pipeline before inference:

1. **Grayscale conversion.** The image is converted to a single-channel representation.
2. **Otsu binarization.** An automatic threshold separates foreground (ink) from background, removing noise and uneven lighting.
3. **Background detection.** Border pixels are sampled; if the majority are white (> 128), the image is inverted to ensure dark-on-light convention.
4. **Aspect-ratio-preserving resize.** The image is scaled to fit within 128×512 pixels (height $\times$ width) without distortion.
5. **Normalization.** Pixel values are mapped to $[0, 1]$.
6. **Mask generation.** A boolean mask marks valid pixels (`False`) vs. padding (`True`).

During training, data augmentation applies random scaling with a factor uniformly sampled from $[0.7, 1.4]$.

6 Training

6.1 Objective

The model is trained with cross-entropy loss over the vocabulary:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T_i} \log P(y_t^{(i)} \mid y_{<t}^{(i)}, \mathbf{x}^{(i)}) \tag{5}$$

where $\mathbf{x}^{(i)}$ is the input image, $y_t^{(i)}$ is the ground-truth token at step t, and padding positions are excluded.

6.2 Optimization

Table 7: Training hyperparameters.

Parameter	Value
Optimizer	Adam (β_1=0.9, β_2=0.999)
Initial learning rate	1×10^{-4}
Weight decay	1×10^{-4}
LR scheduler	ReduceLROnPlateau
Patience	10 epochs
Decay factor	0.5
Gradient clipping	Max norm = 5.0
Mixed precision	FP16 (`torch.cuda.amp`)
Batch size	64
Max batch pixels	4,000,000
Epochs (planned / actual)	300 / 230

6.3 Learning Rate Schedule

The learning rate was automatically reduced by ReduceLROnPlateau when validation loss plateaued:

Table 8: Learning rate decay events.

Epoch	Learning Rate
0	1.00×10^{-4}
82	5.00×10^{-5}
131	2.50×10^{-5}
176	1.25×10^{-5}
211	6.25×10^{-6}

Training was stopped after epoch 230 due to diminishing returns. The best checkpoint was selected at epoch 194.

6.4 Hardware

Training was conducted on a single NVIDIA RTX 5060 Ti with 16 GB VRAM. Total training time was approximately 48 hours.

7 Inference

7.1 Greedy Decoding

The default inference strategy selects the highest-probability token at each step:

Algorithm 1 Greedy Decoding

Require: Encoder features $\mathbf{F}$, mask $\mathbf{M}$, max length L

$\mathbf{y} \leftarrow [\texttt{SOS}]$
for $t = 1$ **to** L **do**
  $\textbf{logits} \leftarrow \text{Decoder}(\mathbf{F}, \mathbf{M}, \mathbf{y})$
  $y_t \leftarrow \arg\max(\textbf{logits}_t)$
  if $y_t = \texttt{EOS}$ **then**
    break
  end if
  Append y_t to $\mathbf{y}$
end for
return $\mathbf{y}$ (excluding SOS)

7.2 Beam Search Decoding

For higher accuracy, beam search maintains k candidate sequences and scores them with a length-normalized log-probability:

$$\text{score}(\mathbf{y}) = \frac{\log P(\mathbf{y} \mid \mathbf{x})}{\left(\frac{5+|\mathbf{y}|}{6}\right)^{\alpha}} \tag{6}$$

where $\alpha = 1.0$ is the length penalty and $k = 10$ is the beam width.

8 Results

8.1 Expression Recognition Rate

The primary metric is the Expression Recognition Rate (ExpRate): the fraction of test samples for which the predicted LaTeX sequence exactly matches the ground truth.

Table 9: Performance comparison on the CROHME test set.

Model	**Parameters**	**ExpRate (%)**
CoMER (original)	∼20 M	59.33
Mini-CoMER (ours)	6.39 M	47.12

Mini-CoMER achieves 79.4% of CoMER's accuracy with only 32% of its parameters, confirming that significant compression is possible with moderate accuracy trade-off. The best checkpoint was obtained at epoch 194 of 230 trained epochs.

8.2 Efficiency

Table 10: Efficiency comparison.

Metric	**CoMER**	**Mini-CoMER**
Parameters	∼20 M	6.39 M
Checkpoint size	∼80 MB	26 MB
Parameter reduction	—	68%

9 System Architecture

MathSnap-AI is deployed as a two-tier web application: a Python backend for model inference and a React frontend for user interaction.

9.1 Backend

The backend is a FastAPI application deployed on Hugging Face Spaces via Docker.

Table 11: REST API endpoints.

Method	Endpoint	Description
`POST`	`/predict`	Accepts a multipart image upload; returns a JSON object `{"latex": "..."}` with the predicted LaTeX string.
`GET`	`/health`	Returns backend status, device type, vocabulary size, and parameter count.

Inference pipeline:

1. Receive image via multipart form data.
2. Preprocess (Section 5).
3. Run greedy decoding with $L_{\max} = 150$.
4. Decode token indices to LaTeX string via vocabulary lookup.
5. Return JSON response.

Deployment stack:

- Runtime: Python 3.10, Uvicorn, FastAPI
- ML framework: PyTorch 2.5.1
- Image processing: OpenCV, Pillow
- Container: Docker (Python 3.10-slim base)
- Port: 7860 (Hugging Face Spaces standard)

9.2 Frontend

The frontend is a React 18 single-page application built with TypeScript and Vite, hosted on Vercel.

Table 12: Frontend technology stack.

Technology	Purpose
React 18 + TypeScript	UI framework
Vite 6.3	Build tool
Tailwind CSS 4.1	Styling
KaTeX 0.16	LaTeX rendering
Radix UI / shadcn	Component library
Recharts 2.15	Data visualization
Motion 12	Animations

Pages:

- **Home.** Landing page with feature highlights.
- **Convert.** Core functionality: drag-and-drop image upload with a side-by-side LaTeX code editor and live KaTeX preview.

- **Dashboard.** Analytics with four tabs: overview KPIs, dataset statistics, model architecture visualization, and training metrics.
- **Settings.** Theme preferences (light/dark mode).

Responsive breakpoints:

- Desktop ($\geq$ 1024 px): three-panel layout.
- Tablet (768–1024 px): compact upload strip with 50/50 code/preview split.
- Mobile ($<$ 768 px): stacked layout with tab switching.

10 Project Structure

Table 13: Key files and directories.

Path	Description
`models/model.py`	Top-level CoMER class, forward pass, greedy and beam search decoding, `build_model` factory
`models/encoder.py`	DenseNet encoder with 2D positional encoding
`models/decoder.py`	Transformer decoder, word embeddings, output projection
`models/pos_enc.py`	`WordPosEnc` (1D) and `ImgPosEnc` (2D)
`models/transformer/`	Attention, ARM, and decoder layer implementations
`data/vocab.py`	Vocabulary class (encode/decode, 114 tokens)
`config.py`	All hyperparameters as Python dataclasses
`backend/main.py`	FastAPI server and preprocessing pipeline
`checkpoints/`	Trained model weights (`model_weights.pt`, 26 MB)
`deployment/huggingface/`	Dockerfile and requirements for HF Spaces
`src/`	React frontend source code

11 Limitations and Future Work

- **Accuracy gap.** Mini-CoMER's ExpRate of 47.12% trails CoMER's 59.33%. Future work could explore knowledge distillation from a full-size CoMER teacher to recover accuracy while retaining the compact architecture.
- **Vocabulary coverage.** The 114-token vocabulary covers common expressions but lacks support for advanced notation (e.g., matrices, piecewise functions, multi-line equations). Expanding the vocabulary and retraining on a more diverse corpus would improve generality.
- **Beam search latency.** Beam search with k=10 is significantly slower than greedy decoding. Speculative decoding or parallel beam evaluation could reduce this gap.
- **Online adaptation.** The current system has no mechanism for user corrections to improve future predictions. An active learning loop could leverage user edits as training signal.

12 Conclusion

MathSnap-AI demonstrates that effective handwritten mathematical expression recognition is achievable with a compact, deployable model. By reducing the CoMER architecture to 6.39 M

parameters while retaining its coverage-aware attention mechanism, Mini-CoMER delivers practical recognition capability on consumer hardware. The complete system—spanning image preprocessing, neural inference, REST API, and responsive web frontend—provides an end-to-end solution for converting handwritten mathematics to LaTeX.

Acknowledgments

This project builds upon the CoMER architecture. The CROHME dataset is provided by the TC-11 online recognition group. See `ATTRIBUTIONS.md` in the project repository for complete attribution details.